Introduction to Signal Processing– Spectrograms and MFCCs

January 28, 2022

1 Introduction to Signal Processing— Spectrograms and MFCCs

We follow the playlist: https://www.youtube.com/playlist?list=PLmZlBIcArwhN8nFJ8VL1jLM2Qe7YCcmAb. Other references: https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html http://practicalcryptography.com/miscellaneous/machine-learning/guide-mel-frequency-cepstral-coefficients-mfccs/ https://wiki.hydrogenaud.io/index.php?title=Pre-emphasis

We will make heavy use of librosa, a package for music and audio analysis.

1.1 Table of Contents:

Preamble

[86]:
```
from scipy import signal
from scipy.io import wavfile    # load wave files
import librosa                  # package for music and audio analysis
import librosa.display
import IPython.display as ipyd
import matplotlib.pyplot as plt
import numpy as np
```

2 1. Fast Fourier Transform (FFT)

2.0.1 Time series and sample rate

Consider a function $x(t)$ of time, of duration T (in second). We wish to represent it by a time series $\{x_n\}$. To this end, we sample each second f_s times (i.e. 1 second contains f_s data points), so that the time step is $\delta t = 1/f_s$. f_s is the **sample rate** in Hz; a typical choice is $f_s = 44100$ Hz. Then $x_n = x(t = n\delta t), n = 0, 1, \ldots, N-1$.

2.0.2 Discrete Fourier Transform (DFT)

The **discrete Fourier transform (DFT)** $\{X_k\}$ of time series $\{x_n\}$ is defined as[1]

$$X_k\{x_n\} \equiv \sum_{n=0}^{N-1} x_n \exp\left(-2\pi i \left(\frac{k}{N}\right) n\right) = \sum_{n=0}^{N-1} x_n \exp\left(-2\pi i \left(\frac{k}{T}\right)\left(\frac{n}{f_s}\right)\right),\ k = 0, \ldots, N-1. \quad (1)$$

From the last equality, it is clear (since $n/f_s = n\delta t = t$) that the discrete Fourier frequencies are

$$f_k = \frac{k}{T} = \frac{k}{N} f_s,\ k = 0, \ldots, N-1. \quad (2)$$

Note that if the signal is *purely real*, then the DFT is *even symmetric*:

$$x_n \in \mathbb{R}\ \forall n \in \{0, \ldots, N-1\} \quad \Rightarrow \quad X_k = X^*_{-k \bmod N}\ \forall k \in \{0, \ldots, N-1\}. \quad (3)$$

In this case (and consider N even), the DFT is completely specified by $X_0, X_1, \ldots, X_{N/2}$, where X_0 and $X_{N/2}$ are real, $X_1, \ldots, X_{N/2-1}$ are complex.

A **fast Fourier transform (FFT)** is an algorithm that more efficiently performs a DFT: the computational complexity is $O(N \log(N))$ instead of the brute-force $O(N^2)$. See Wikipedia.

[1] In signal processing, we usually first apply a window function on x_n before taking the DFT X_k; see Section 4 below. It is kept implicit here.

2.0.3 The Nyquist frequency and Nyquist rate

With a given sample rate f_s, what is the highest frequency in a signal that we can detect?

Suppose we have a sine wave. To measure its frequency, we need at least one cycle. A cycle contains a positive and a negative stage, and we need to detect both of them. That is, each cycle must contain at least two samples (data points). The extreme case is when the samples are taken at the peak and the trough of the wave, in which case the period of the wave equals twice the inverse $1/f_s$ of sample rate.

In other words, with a given sample rate f_s, the *highest* frequency a system can recreate is the **Nyquist frequency**,

$$f_{\text{Nyquist frequency}} = 2 \times f_s. \quad (4)$$

Conversely, to recreate a sine wave of frequency f, the *minimum* sample rate required is the **Nyquist rate**,

$$f_{\text{Nyquist rate}} = \frac{1}{2} \times f. \quad (5)$$

2.1 Example: sine wave

[152]:
```
f_s = 16000  # sample rate in Hz. 1 second contains f_s data points
# consider a perfect sine wave of frequency f_0 (Hz)
```

```
N = 3 * f_s  # number of data points in the file, N = length (seconds) * sample␣
 ↪rate
n = np.arange(N)
f_0 = 300  # frequency of our sine wave in Hz
x = np.sin(2*np.pi*(f_0/f_s)*n)

ipyd.Audio(rate = f_s, data = x)
```

[152]:
```
<IPython.lib.display.Audio object>
```

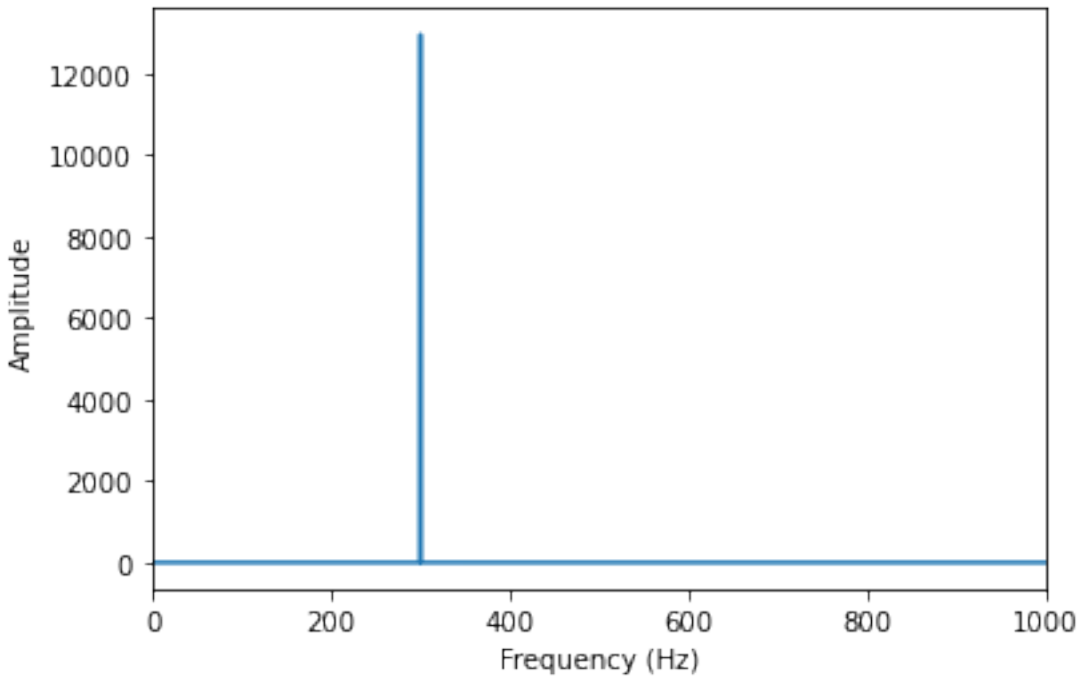
2.1.1 Plot spectrum

Use X = np.fft.fft(x).

[9]:
```
len(X)
```

[9]:
```
48000
```

[153]:
```
N = len(x)
# convolute the time series x by the Hamming window function to smooth out the␣
 ↪DFT and reduce edge effect
# see the last section
w = np.hamming(N)
# FFT
X = np.fft.fft(x*w)
# discrete Fourier frequencies are (k/N)*f_s, k=0, ..., N-1
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.abs(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude")
plt.xlim([0, 1000]);
```



2.2 Example: violin_A.wav

2.2.1 Import .wav file

Use wavfile.read(filename).

[154]:
```
filename = 'violin_A.wav'
# extract sample rate (e.g. 44100Hz) and audio as time series
f_s, x = wavfile.read(filename)
```

[14]:
```
f_s
```

[14]:
```
44100
```

[25]:
```
x
```

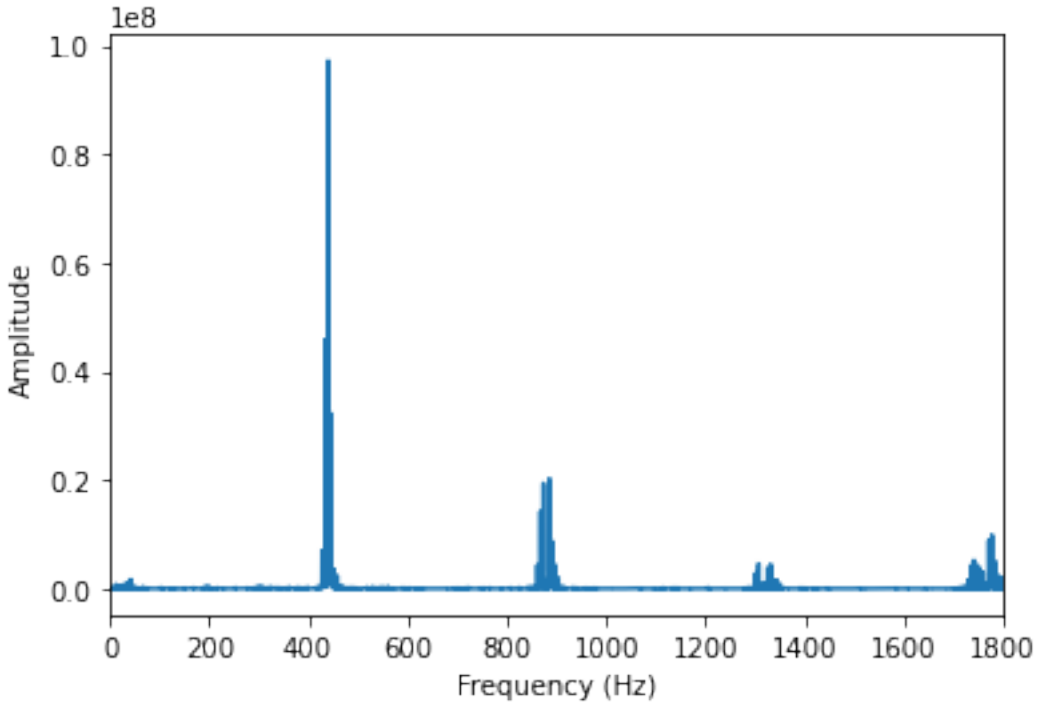
[25]:
```
array([ 0,  0,  0, ...,  3, -3,  2], dtype=int16)
```

[26]:
```
ipyd.Audio(rate=f_s, data=x)
```

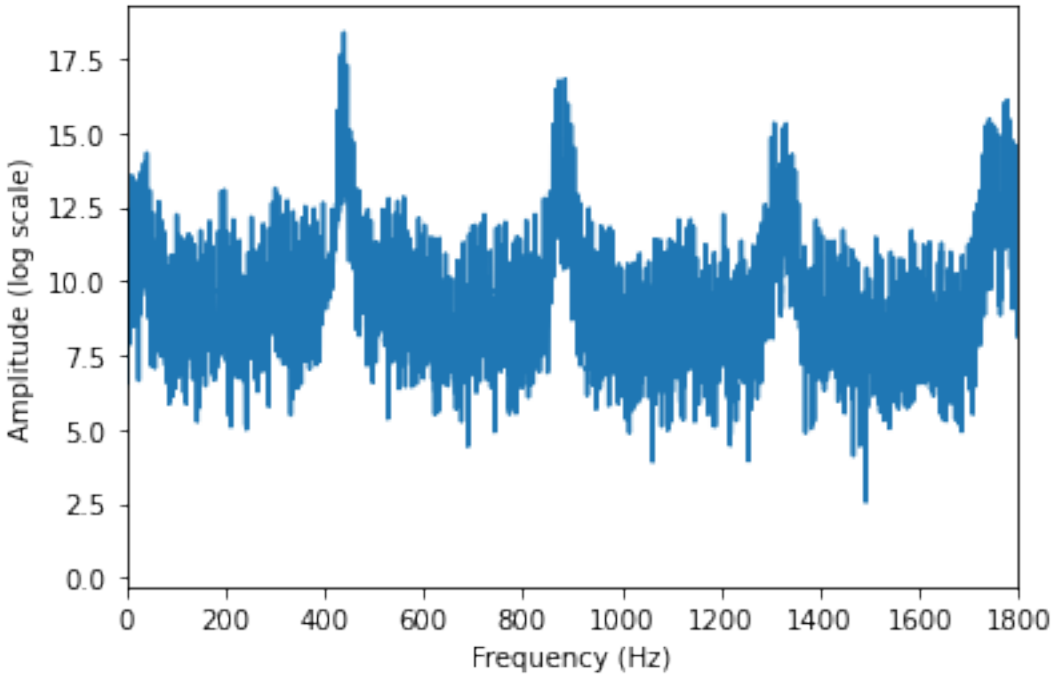
[26]:
```
<IPython.lib.display.Audio object>
```

2.2.2 Plot spectrum

[155]:
```
N = len(x)
# convolute the time series x by the Hamming window function (Google it) to␣
 ↪reduce edge effect
w = np.hamming(N)
# FFT
X = np.fft.fft(x*w)
# discrete Fourier frequencies are (k/N)*f_s
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.abs(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude")
plt.xlim([0, 1800]);
```



[156]:
```
# log plot
# discrete Fourier frequencies are (k/N)*f_s
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.log(np.abs(X)))
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude (log scale)")
plt.xlim([0, 1800]);
```



They obviously show the fundamental at ~440Hz and some overtones.

3 2. Short-Time Fourier Transform (STFT) & Spectrogram

In the last section, we performed a DFT on the entire signal (audio file). However, when the signal is non-stationary (which is almost always the case), this approach is not very useful. For example, if the signal is a musical passage, we might want to analyse the frequencies of the musical notes and their duration, each of which would have its own, different DFT.

Instead, it is more useful to perform the **short-time Fourier transform (STFT)**. The signal is broken into *overlapping* time frames (the overlapping is to reduce boundary effects). Each frame is then Fourier-transformed. Denote the STFT of a signal x_n as $STFT\{x_n\}(\omega, m)$, where ω is the (discrete) Fourier frequency, and $m = 0, \ldots,$ n_frames labels the frame.

Parameter to specify: 1. n_fft = frame_size * sample_rate = number of data points in each frame (frame_size is typically 25ms for human voice) 2. hop_length = stride_in_second * sample_rate = stride in unit of timestep $1/f_s$ (stride is typically 10ms)

3.0.1 Spectrogram

The **spectrogram** representation of the Power Spectral Density of the signal is the magnitude squared of the STFT.

3.0.2 librosa

To perform an STFT, we use librosa.stft(x, n_fft=n_fft, hop_length=hop_length).

This function takes in a **real** signal x, as well as the parameters n_fft and hop_length. Since x is a *real* signal, as discussed in the last section, the even symmetry property means that the DFT of each frame is completely specified by the first half of the discrete Fourier modes $X_0, X_1, \ldots, X_{\text{n\_fft}/2}$; that's a total of 1+n_fft/2 modes.

As such, this function returns a complex matrix of shape = (1+n_fft/2, n_frames).

To convert the Fourier mode labels back to frequencies, we simply use (2) from the last section:

$$f_k = \frac{k}{\text{n\_fft}} f_s, \; k = 0, \ldots, 1 + \text{n\_fft}/2. \tag{6}$$

Note that k runs in the first half of the discrete Fourier modes, since the other half is redundant and that's what the function returns.

To plot the spectrogram, we use the librosa.display.specshow() function. Be sure to specify the sample rate sr parameter (default=22050).

3.1 Example: violin_A.wav

As a sanity check: the spectrogram should clearly show highest amplitude (dB) in the fundamental at ~440Hz, and sub-peaks at the overtones.

[157]:
```
filename = 'violin_A.wav'
# filename = 'bruckner.wav'
# extract sample rate (e.g. 44100Hz) and signal
f_s, x = wavfile.read(filename)
x = x / 1.0  # convert to float
ipyd.Audio(rate=f_s, data=x)
```

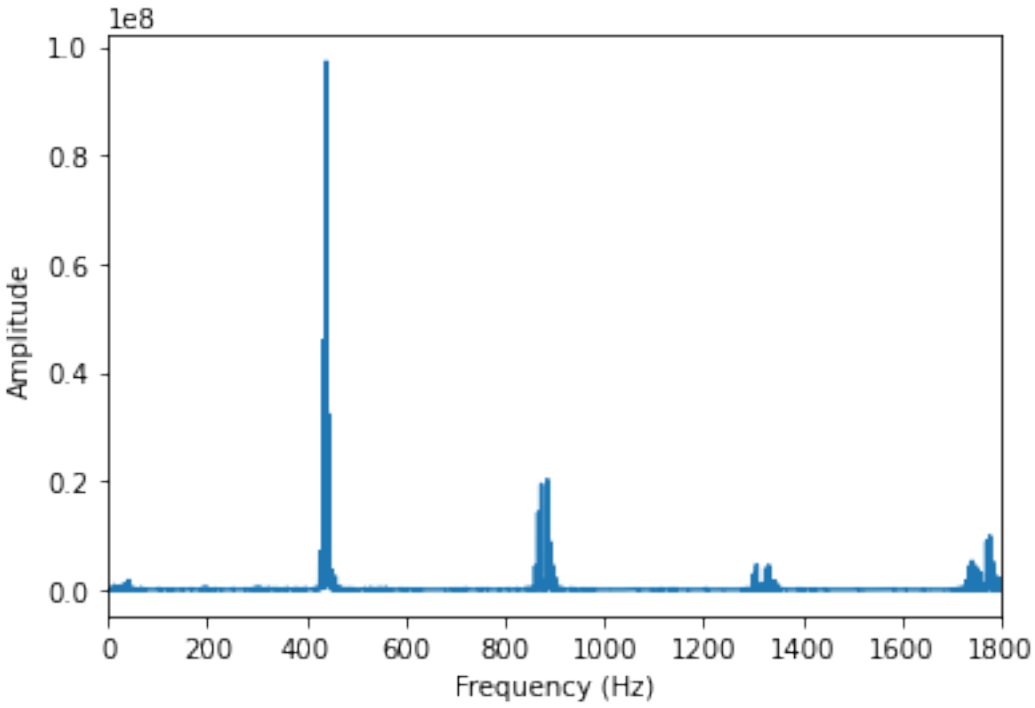
[157]:
```
<IPython.lib.display.Audio object>
```

[11]:
```
f_s
```

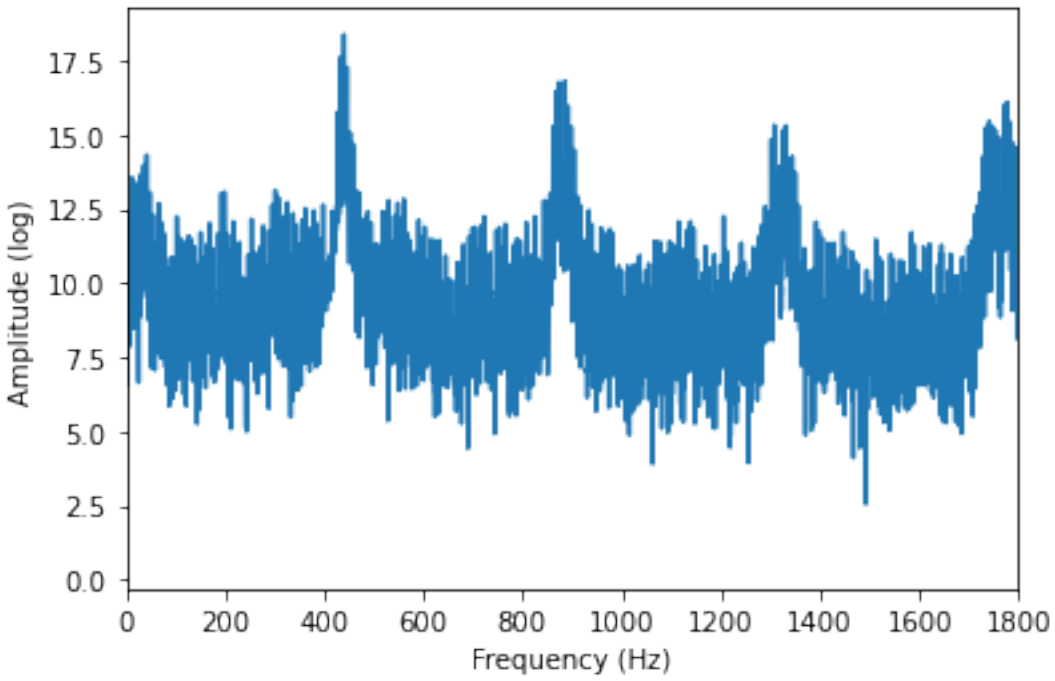
[11]:
```
44100
```

3.1.1 Plot spectrum

[158]:
```
N = len(x)
w = np.hamming(N)
# FFT
X = np.fft.fft(x*w)
# discrete Fourier frequencies are (k/N)*f_s
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.abs(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude")
plt.xlim([0, 1800]);
```



[159]:
```
# log plot
# discrete Fourier frequencies are (k/N)*f_s
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.log(np.abs(X)))
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude (log)")
plt.xlim([0, 1800]);
```



3.1.2 Spectrogram

[160]:
```
# Short-time fast Fourier transform (STFFT)
n_fft = int(0.025*f_s)  # 25ms
hop_length = int(0.01*f_s)  # 10ms
stfft = librosa.stft(x, n_fft=n_fft, hop_length=hop_length)
abs_stfft = np.abs(stfft)
```

[15]:
```
abs_stfft.shape
```

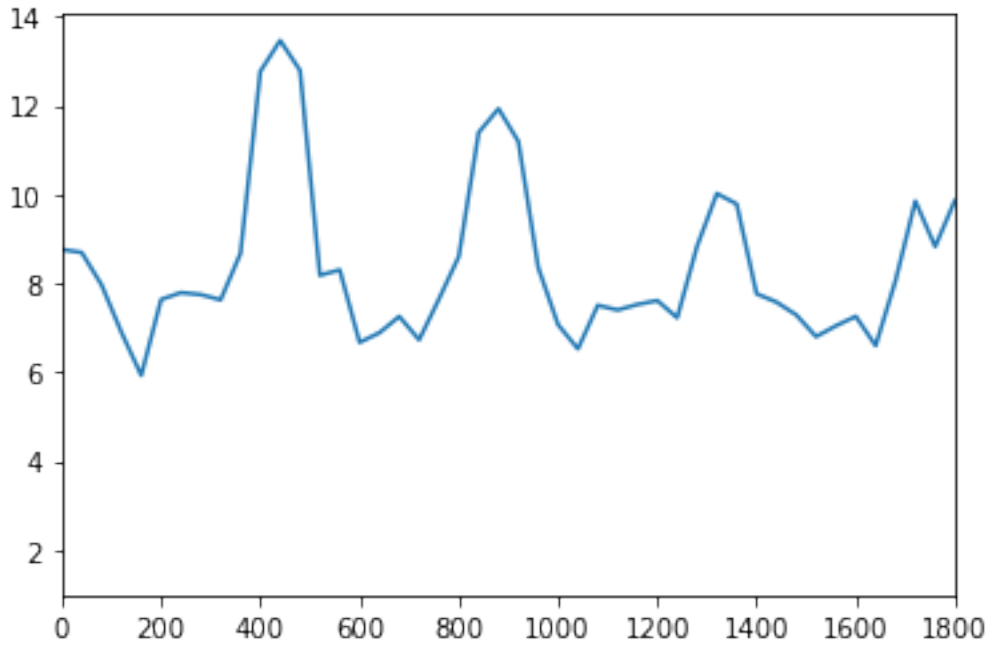
[15]:
```
(552, 620)
```

[16]:
```
# number of strided windows
len(x)/f_s/0.01
```

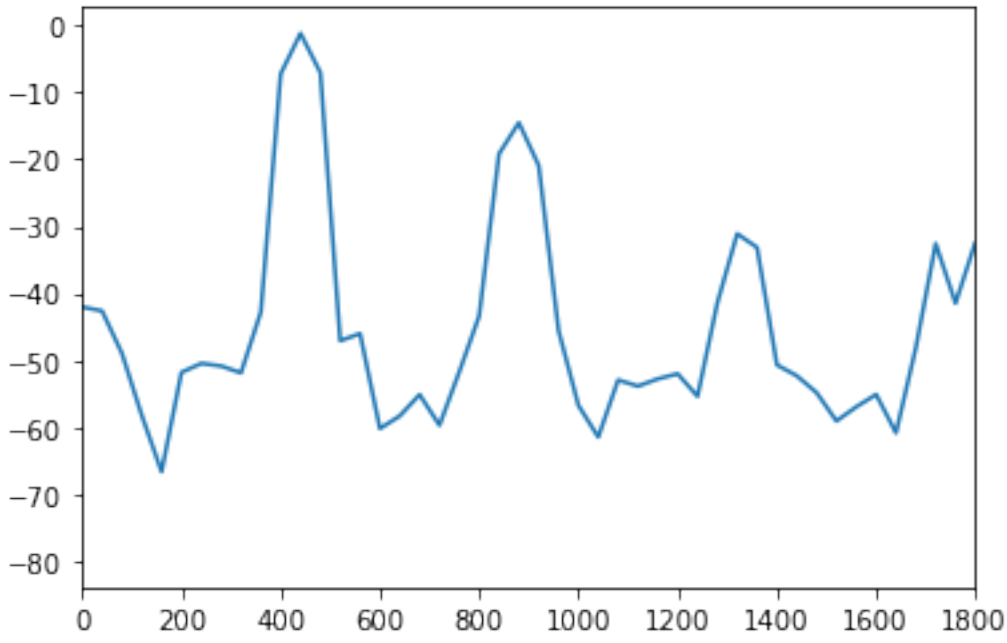
[16]:
```
619.9727891156463
```

[20]:
```
abs_stfft_freq = np.arange(0,int(1+n_fft/2)) * f_s / n_fft
```

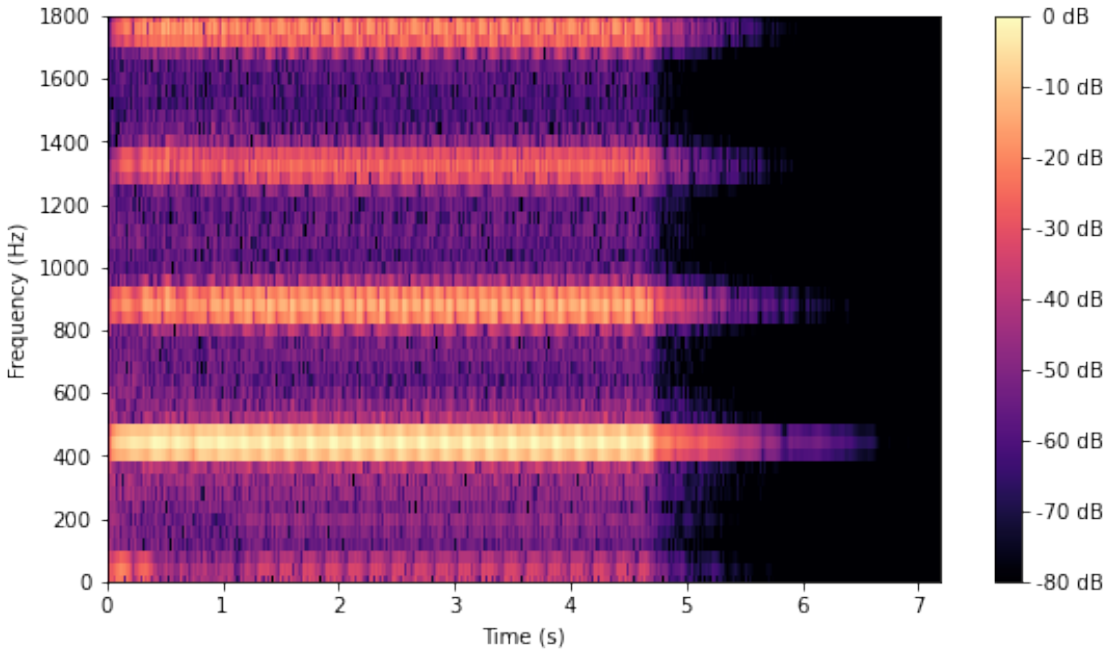
[161]:
```
plt.plot(abs_stfft_freq, np.log(abs_stfft[:,100]))
plt.xlim([0,1800]);
```



[162]:
```
plt.plot(abs_stfft_freq, librosa.amplitude_to_db(abs_stfft, ref=np.max)[:,100])
plt.xlim([0,1800]);
```



[163]:
```
# Plot spectrogram
fig, ax = plt.subplots(figsize=(9,5))
img = librosa.display.specshow(librosa.amplitude_to_db(abs_stfft, ref=np.max),␣
 ↪sr=f_s,
                               y_axis='hz', x_axis='time', ax=ax, fmax=6000)
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
plt.ylim([0, 1800])
fig.colorbar(img, ax=ax, format="%2.0f dB");
```



Indeed, peaks at the fundamental (~440Hz) and overtones.

3.2 Example: bruckner.wav

[164]:
```
# filename = 'violin_A.wav'
filename = 'bruckner.wav'
# extract sample rate (e.g. 44100Hz) and signal
f_s, x = wavfile.read(filename)
x = x / 1.0  # convert to float
ipyd.Audio(rate=f_s, data=x)
```

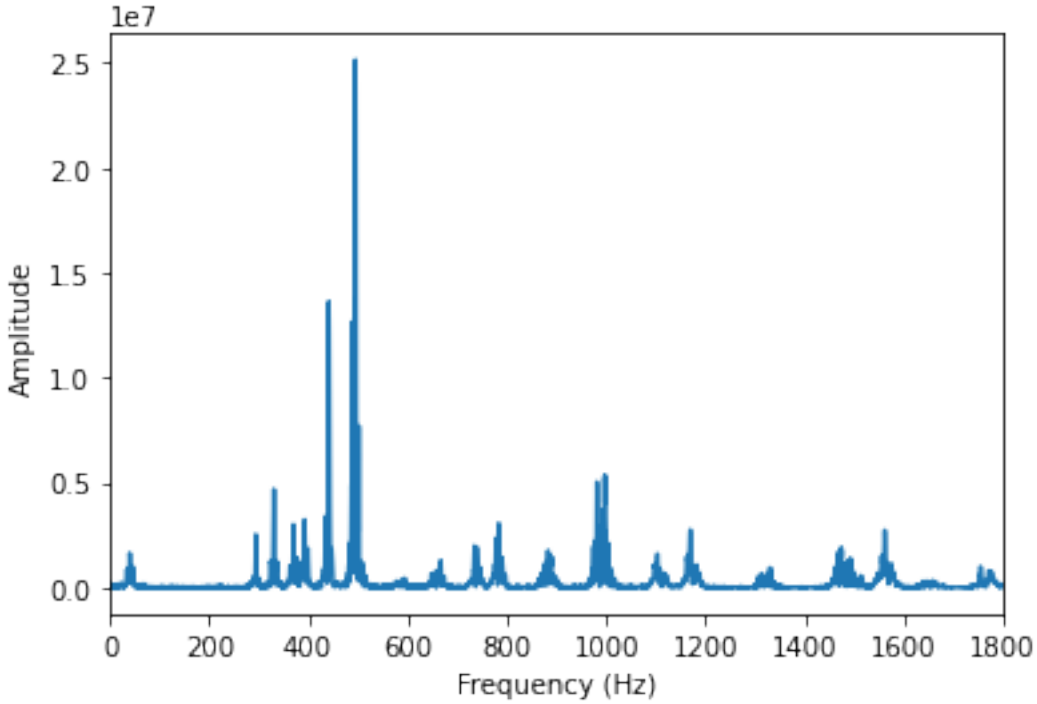
[164]:
```
<IPython.lib.display.Audio object>
```

[28]:
```
f_s
```

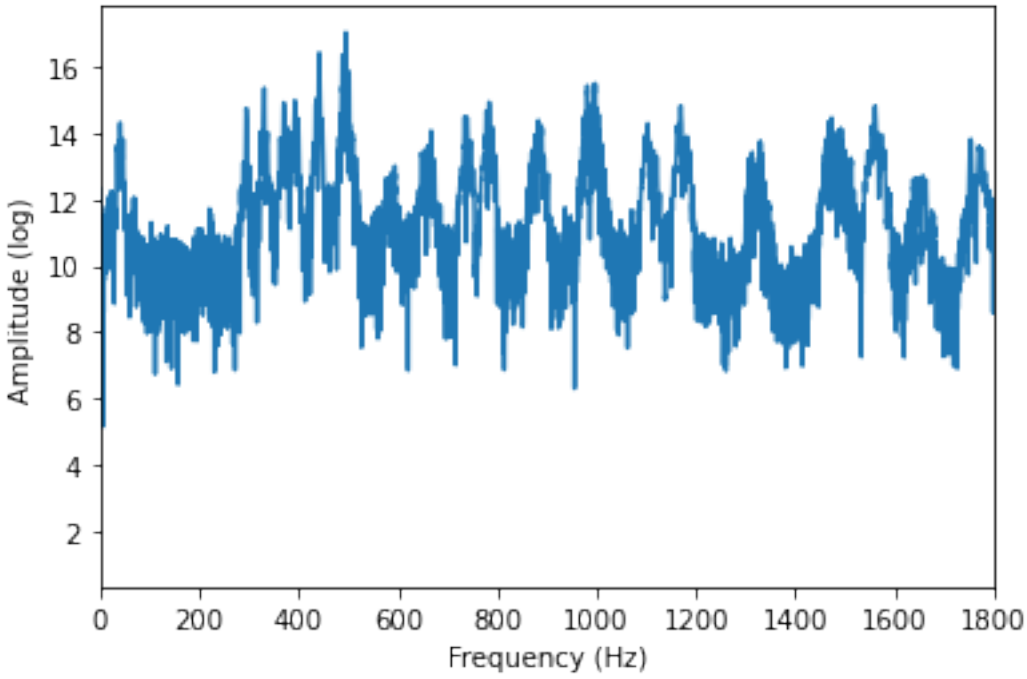
[28]: 44100

3.2.1 Plot spectrum

[165]:
```
N = len(x)
w = np.hamming(N)
# FFT
X = np.fft.fft(x*w)
# discrete Fourier frequencies are (k/N)*f_s
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.abs(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude")
plt.xlim([0, 1800]);
```



[166]:
```
# log plot
# discrete Fourier frequencies are (k/N)*f_s
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.log(np.abs(X)))
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude (log)")
plt.xlim([0, 1800]);
```



DFT is not very informative.

3.2.2 Spectrogram

[167]:
```
# Short-time fast Fourier transform (STFFT)
n_fft = int(0.025*f_s)  # 25ms
hop_length = int(0.01*f_s)  # 10ms
stfft = librosa.stft(x, n_fft=n_fft, hop_length=hop_length)
abs_stfft = np.abs(stfft)
```

[32]:
```
abs_stfft.shape
```

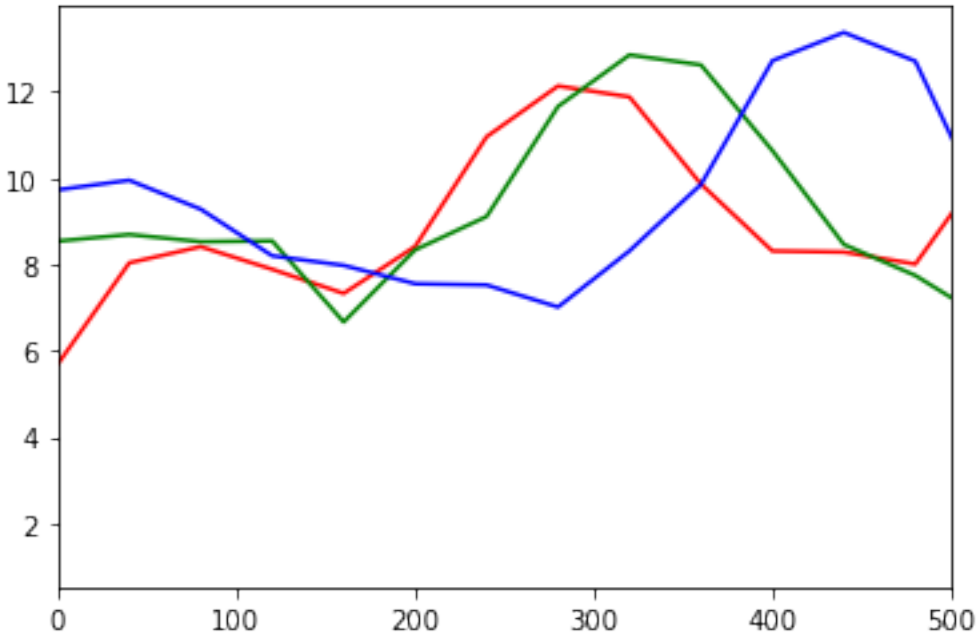
[32]:
```
(552, 514)
```

[33]:
```
# number of strided windows
len(x)/f_s/0.01
```

[33]:
```
513.1609977324264
```

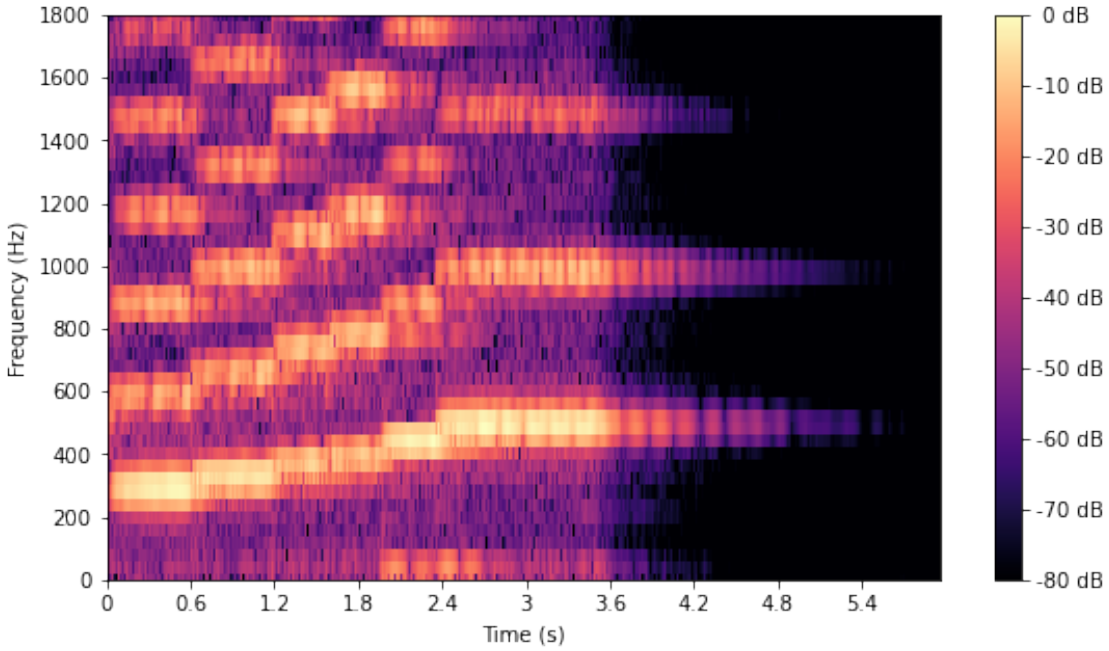
[168]:
```
abs_stfft_freq = np.arange(0,int(1+n_fft/2)) * f_s / n_fft
```

[169]:
```
plt.plot(abs_stfft_freq, np.log(abs_stfft[:,5]), color='red')
plt.plot(abs_stfft_freq, np.log(abs_stfft[:,100]), color='green')
plt.plot(abs_stfft_freq, np.log(abs_stfft[:,200]), color='blue')
plt.xlim([0,500]);
```



Ascending notes.

[170]:
```
# Plot spectrogram
fig, ax = plt.subplots(figsize=(9,5))
img = librosa.display.specshow(librosa.amplitude_to_db(abs_stfft, ref=np.max),␣
 ↪sr=f_s,
                               y_axis='hz', x_axis='time', ax=ax, fmax=6000)
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
plt.ylim([0, 1800])
fig.colorbar(img, ax=ax, format="%2.0f dB");
```



Clearly the Bruckner Rhythm.

[ ]:

[ ]:

4 3. Log Mel-Scale Spectrogram

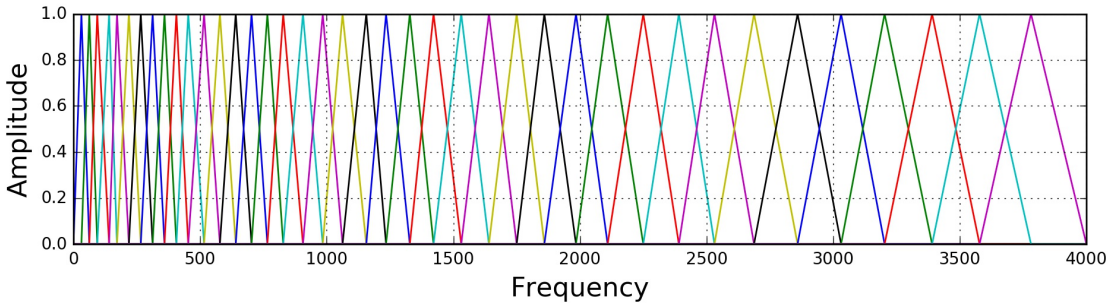
4.0.1 Mel scale

Our ears are more sensitive to lower frequencies than higher frequencies. This is captured by the **Mel scale**. This perceived frequency, empirically given by the Mel scale, is related *logarithmically* to the linear frequency:

$$m = 2595 \log_{10}\left(1 + \frac{f}{700}\right). \tag{7}$$

4.0.2 Mel filter bank

A **filter bank** is an array of bandpass filters that separates the input signal into multiple components, each carrying a single frequency sub-band of the original signal. For current purpose, each filter is triangular, having a response of 1 at the centre frequency, and decrease linearly towards 0 untill it reaches the centre frequencies of the two adjacent filters where the response is 0. In a **Mel filber bank**, the centre frequencies are evenly spaced in the *Mel scale*. See the following figure:



Source: https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html

4.0.3 Log Mel-scale spectrogram

To convert a spectrogram in linear frequency to a Mel-scale one, for each frame in the STFT, we weigh the discrete Fourier spectrum by the Mel filter bank, i.e. multiply the two values at each frequency.

Further take the log to get the log Mel spectrogram; human ears hear loudness on a log scale.

4.0.4 librosa

Filter banks: librosa.filters.mel(sr, n_fft, n_mels=128, fmin=0.0) Mel spectrogram: librosa.feature.melspectrogram(y=None, sr=22050, n_fft=2048, hop_length=512). Returns an array of shape = (n_mels, n_frames) (VS spectrogram which returns an array of shape = (1+n_fft/2, n_frames)).

4.0.5 Mel filter banks: visualisation

[78]:
```
f_s = 16000
filterbanks = librosa.filters.mel(sr=f_s, n_fft=512, n_mels=20, fmin=0.0)
```

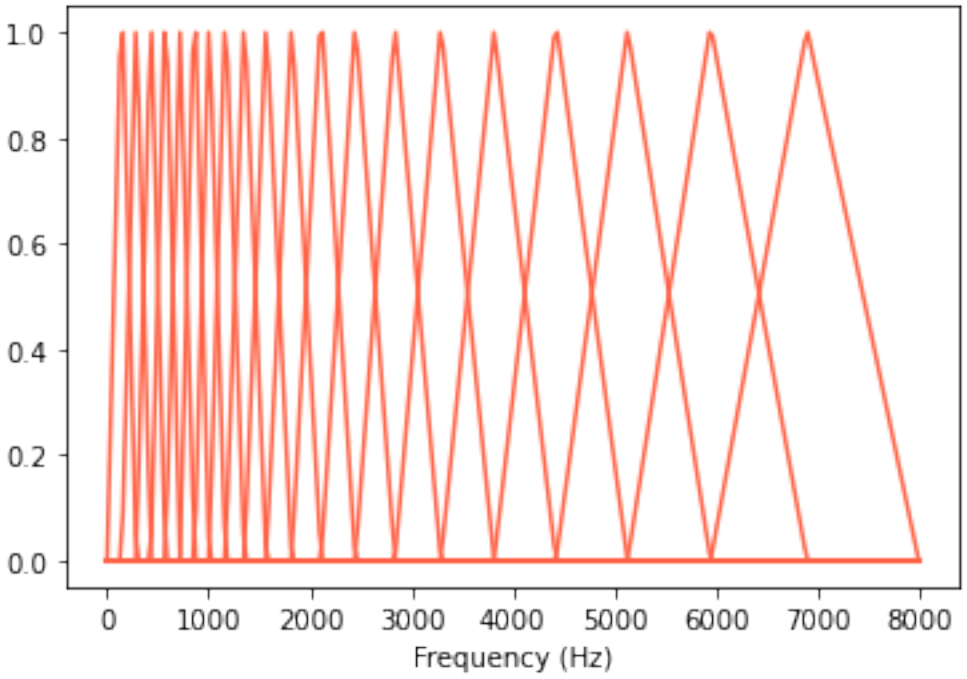
[79]:
```
filterbanks.shape
```

[79]:
```
(20, 257)
```

[80]:
```
# normalise the triangular filters for visualisation
filterbanks /= np.expand_dims(np.max(filterbanks, axis=-1), axis=-1)
```

[81]:
```
filterbanks_freq = np.arange(filterbanks.shape[1]) / 512 * f_s
```

[91]:
```
plt.plot(filterbanks_freq, filterbanks.T, color='tomato');
plt.xlabel('Frequency (Hz)');
```



4.0.6 Mel spectrogram

[103]:
```
# filename = 'violin_A.wav'
filename = 'bruckner.wav'
# extract sample rate (e.g. 44100Hz) and signal
f_s, x = wavfile.read(filename)
x = x / 1.0  # convert to float
ipyd.Audio(rate=f_s, data=x)
```

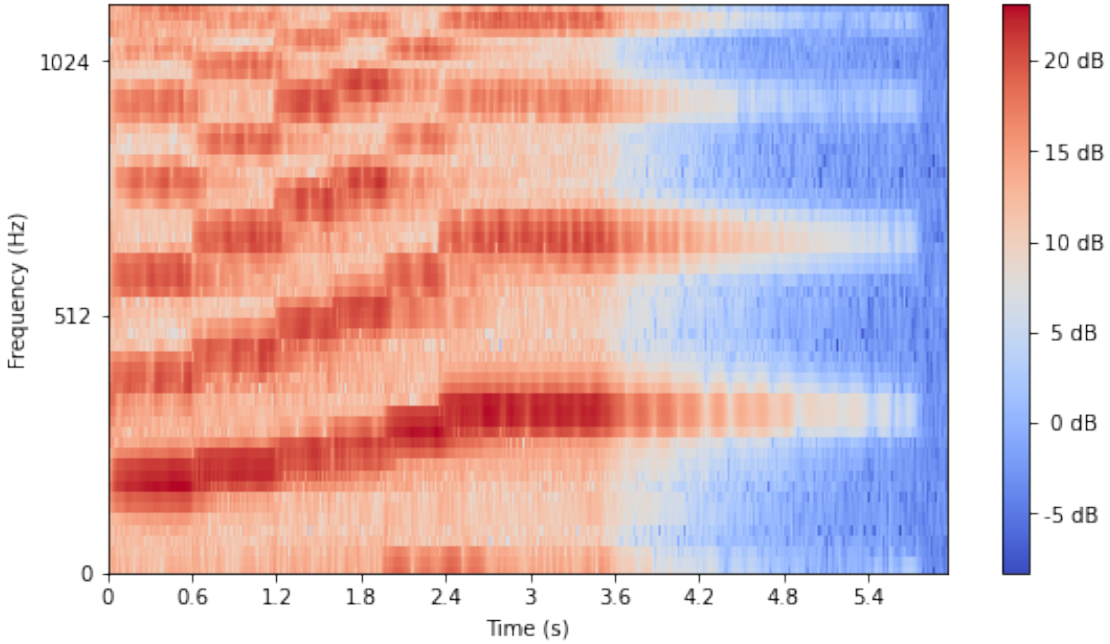
[103]: <IPython.lib.display.Audio object>

[108]:
```
# construct Mel spectrogram
n_fft = int(0.025*f_s)  # 25ms
hop_length = int(0.01*f_s)  # 10ms
mel_spec = librosa.feature.melspectrogram(x, sr=f_s, n_fft=n_fft,␣
 ↪hop_length=hop_length, n_mels=128)
log_mel_spec = np.log(mel_spec)
```

[109]:
```
# shape=(n_mels, n_frames)
log_mel_spec.shape
```

[109]: (128, 514)

[112]:
```
# Plot Mel spectrogram
fig, ax = plt.subplots(figsize=(9,5))
img = librosa.display.specshow(log_mel_spec, sr=f_s,
                               y_axis='mel', x_axis='time', ax=ax, fmax=6000);
plt.xlabel('Time (s)');
plt.ylabel("Frequency (Hz)");
plt.ylim([0, 1200])
fig.colorbar(img, ax=ax, format="%2.0f dB");
```



[ ]:

5 4. Windowing, Pre-Emphasis Filter, Mel-Frequency Cepstral Coefficients (MFCC)

5.0.1 Windowing

A **window function** $w(t)$ (also known as an apodisation function or tapering function) is a mathematical function that is zero-valued outside of some chosen interval, normally symmetric around the middle of the interval, usually near a maximum in the middle, and usually tapering away from the middle.

In signal processing, we weigh a signal x_n by the discretisation of a window function w_n, i.e. we compute the point-wise product

$$\tilde{x}_n \equiv x_n \cdot w_n. \tag{8}$$

The discrete Fourier transform X_k defined in (1) in Section 1 should instead be understood as that of $\tilde{x}_n$.

A benefit of using such a window function is that it reduces the boundary effect of the signal (or a frame thereof in STFT), rendering the resulting DFT smoother.

A popular choice of the window function is the **Hamming function**, which is a special case of a **cosine-sum window function** (see Wikipedia).

5.0.2 Pre-emphasis filter

The **pre-emphasis** y_n of a signal x_n is defined as

$$y_n \equiv x_n - \alpha x_{n-1}, \tag{9}$$

where typically the coefficient takes the value $\alpha = 0.97$.

A **pre-emphasis filter** is a noise reduction technique in which a signal's weaker, higher frequencies are boosted before they are transmitted or recorded onto a storage medium. Upon playback, a **de-emphasis filter** is applied to reverse the process. The result is a higher signal-to-noise ratio; the original frequencies are restored, but noise that was introduced by the storage medium, transmission equipment, or analog/digital conversions is quieter than it would have been if no filtering had been done. Pre- and de-emphasis can be collectively referred to as just **emphasis**.

Source: https://wiki.hydrogenaud.io/index.php?title=Pre-emphasis

The pre-emphasis operation is applied, where applicable, on the signal as the 0-th step, i.e. before weighing it by the window function.

5.0.3 Mel-Frequency Cepstral Coefficients (MFCC)

In obtaining the (log) Mel spectrogram, we used a filter bank which contains overlapping triangular filters. Because of the overlapping, the filter bank energies (represented by the spectrogram) are quite correlated with each other. The **discrete cosine transform (DCT)** decorrelates the energies, which means that the diagonal covariance matrices can be used to model the features. The resulting DCT coefficients are called the **Mel-frequency cepstral coefficients (MFCC)** (compare with the definition of cepstral coefficients in Wikipedia).

The higher DCT coefficients, which represent fast changes in the filter bank energies, degrade performance in **Automatic Speech Recognition (ASR)**. Thus, one typically discards them and **retain only the cepstral coefficients 2-13** to get a small improvement.

References: https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html http://practicalcryptography.com/miscellaneous/machine-learning/guide-mel-frequency-cepstral-coefficients-mfccs/

5.0.4 librosa

MFCC: librosa.feature.mfcc(y=None, sr=22050, n_mfcc=20)

5.0.5 Pre-Emphasis

[121]:
```
def preemphasis(x, alpha=0.97):
    return np.append(x[0], x[1:] - alpha * x[:-1])
```

[113]:
```
filename = 'violin_A.wav'
# filename = 'bruckner.wav'
# extract sample rate (e.g. 44100Hz) and signal
f_s, x = wavfile.read(filename)
x = x / 1.0  # convert to float
ipyd.Audio(rate=f_s, data=x)
```

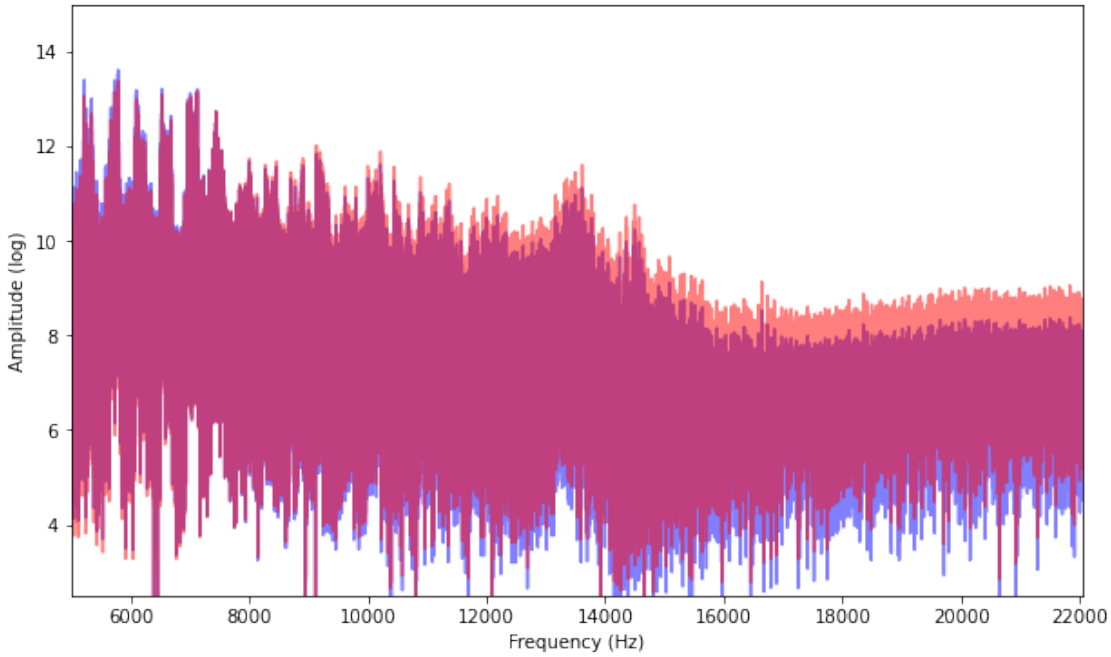
[113]:
```
<IPython.lib.display.Audio object>
```

[137]:
```
N = len(x)
w = np.hamming(N)
# FFT
X = np.fft.fft(x*w)

# log plot of DFT without pre-emphasis
plt.figure(figsize=(10,6))
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.log(np.abs(X)), color='blue',␣
 ↪alpha=0.5)
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude (log)")

# pre-emphasis
x_preem = preemphasis(x)
# FFT
X_preem = np.fft.fft(x_preem*w)

# log plot of DFT WITH pre-emphasis
plt.plot(np.linspace(0, f_s*(N-1)/N, N), np.log(np.abs(X_preem)), color='red',␣
 ↪alpha=0.5)
plt.xlabel('Frequency (Hz)')
plt.ylabel("Amplitude (log)")
plt.xlim([5000, 22050]);
plt.ylim([2.5, 15.]);
```



Indeed, high frequency modes are boosted.

5.0.6 MFCCs

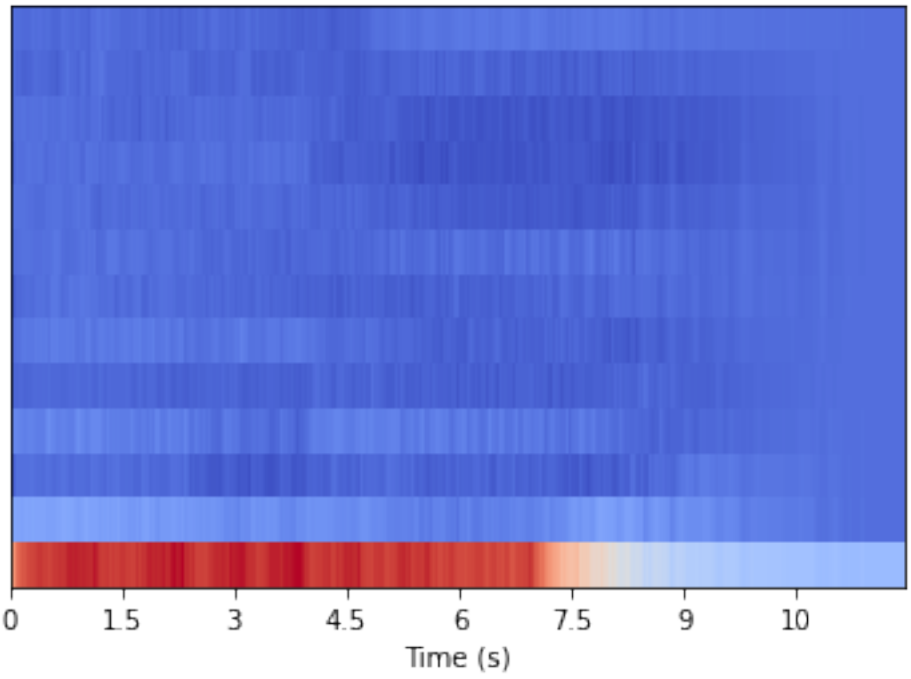
[147]:
```
# filename = 'violin_A.wav'
filename = 'bruckner.wav'
# extract sample rate (e.g. 44100Hz) and signal
f_s, x = wavfile.read(filename)
x = x / 1.0  # convert to float
ipyd.Audio(rate=f_s, data=x)
```

[147]:
```
<IPython.lib.display.Audio object>
```

[151]:
```
# construct MFCCs
n_fft = int(0.025*f_s)  # 25ms
hop_length = int(0.01*f_s)  # 10ms

mfcc = librosa.feature.mfcc(x, sr=f_s, n_mfcc=13, n_mels=24,
                            n_fft=n_fft, hop_length=hop_length,
                            fmin=64, fmax=8000)

librosa.display.specshow(mfcc, x_axis='time');
plt.xlabel('Time (s)');
```



[ ]:

[ ]:

5.1 5. Summary: From Signal to Mel-Frequency Cepstral Coefficients (MFCC)

The general workflow of obtaining a Mel-frequency cepstral coefficients (MFCC) from a signal is as follows.

0. Apply a **pre-emphasis filter**, where applicable, to boost high-frequency signals.
1. Apply a **window function** to smooth out boundary effects, and compute the **short-time Fourier transform (STFT)**. The resulting discrete Fourier coefficients are *complex* numbers.
2. Take the **absolute values** of the Fourier coefficients to get the power spectrum.
3. Apply a **Mel filter bank** and take the **log scale** to reflect how human ears perceive sound. The result is a **log Mel-scale spectrogram**. **MFCC:**
4. Apply a **discrete cosin transform (DCT)** to de-correlate filter bank energies that result from overlapping triangular Mel filters. The DCT coefficients are called the **Mel-frequency cepstral coefficients (MFCC)**.
5. Keep only the 2-13 **MFCCs** for Automatic Speech Recognition (ASR), because the higher MFCCs represent fast changes in the filter bank energies, which degrade performance.

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]: